HW 2 Core: Projectile Clears a Wall

Physics Programming, Week 2

Submission: submit the fragment in the Week 2 core CodeRunner question on the course Moodle page.

Core homework: does a projectile clear a wall?

We launch from ground level with speed v_0, angle θ, and gravity magnitude $g > 0$, with $+x$ toward the wall and $+y$ upward. In the uniform-gravity, no-drag model,

$$v_x = v_0 \cos\theta, \quad t_w = x_w / v_x, \quad y_w = v_0 \sin\theta \, t_w - \tfrac{1}{2} g t_w^2, \quad v_{y,w} = v_0 \sin\theta - g t_w.$$

We require $v_x > 0$ before dividing. We compare y_w with the wall height, and use $v_{y,w}$ to classify rising or falling. Equality in height counts as clearing; zero vertical velocity belongs to the published falling category. The computed free-flight height may be negative: for a non-negative wall height this means failure to clear, not motion physically continuing through the ground.

Physics box: projectile at a wall

A projectile launched from the origin is pulled downward by gravity, so its horizontal speed stays constant while its vertical speed decreases. In uniform gravity with no drag, $v_x = v_0 \cos\theta$ and $v_y(t) = v_0 \sin\theta - gt$, and the height at horizontal distance x_w is $y_w = v_0 \sin\theta \, t_w - \frac{1}{2} g t_w^2$ with $t_w = x_w / v_x$. Here v_0 is the launch speed in m/s, θ the angle above the horizontal in degrees, $g > 0$ the gravity magnitude in m/s^2, x_w and y_w lengths in metres, t_w a time in seconds, and $v_{y,w}$ a vertical velocity in m/s. We take $+x$ toward the wall and $+y$ upward, launch from ground level, and require $v_x > 0$. The model assumes a point mass, flat ground, uniform gravity, and no air resistance. For $v_0 = 20$ m/s, $\theta = 45°$, and $x_w = 10$ m it gives $y_w = 7.5475$ m. A negative computed height means the free-flight parabola crossed the ground before the wall; it is not motion continuing underground. Air drag, spin, wind, and any collision with the wall lie outside this model.

We first write a prediction for one ordinary case and one boundary. Next we implement the scalar fragment, validate its documented domain, and compare the returned labels and Boolean exactly. We check heights approximately using the published tolerance. Finally we explain how the clearance decision and vertical-state decision differ: a projectile can clear while falling. The assignment sheet is the place to look for the precise input names, the tests, the convention for the returned wrapper values, and the submission instructions; neither student-defined functions nor loops are needed.

Task

A projectile starts at the origin with speed `speed_m_s` and launch angle `angle_deg`. Using the analytic, no-drag model, calculate its height at a vertical wall and decide whether it is rising or falling there and whether it clears the wall. Use $+x$ horizontally and $+y$ upward, $g > 0$, and angles converted from degrees to radians. At the wall, $t = x/(v\cos\theta)$, $y = v\sin\theta\, t - gt^2/2$, and $v_y = v\sin\theta - gt$. Equality with the wall counts as clearing; $v_y = 0$ is `"falling"` by contract. We assume uniform gravity and no drag; this does not model impact with the wall or ground.

Expected submission

Submit a fragment only (functions are taught in Week 5). The supplied wrapper returns `height_at_wall, vertical_state, clears`. Validate finite non-Boolean real inputs: the launch angle is in $[0°, 90°)$, speed, wall distance, and wall height are nonnegative, gravity is positive, and horizontal velocity must be positive. (In Julia, `Bool` is a `Real`, so reject it explicitly.) Invalid inputs must `throw(ArgumentError(...))`, not print a warning. Assign exactly `"rising"` or `"falling"` and a `Bool`; print nothing.

```
# supplied: speed_m_s, angle_deg, wall_x_m, wall_height_m, gravity_m_s2
# assign: height_at_wall, vertical_state, clears
```

How is this exercise checked?

When we submit, the public checks walk through rising and falling trajectories, a case that just clears the wall, and a few invalid inputs. Heights are compared approximately with the published tolerance, while the state string and the Boolean are compared exactly, since those are discrete outcomes with nothing to round. The staff checks draw on the same stated domain and simply use other ordinary and boundary values, so they never ask for anything the published contract leaves out.

Advanced (optional)

The wrapper and the `@test` checks belong to the platform, and we do not write them this week. A wrapper is a supplied function that calls the submitted code and returns its results; a test written with `@test` states a claim that has to hold, while `@test_throws` states that an invalid input must be rejected. Reading them shows us how our work is checked the same way every time. The complete public test body is:

```
@testset "week02 core public" begin
    h, state, clears = wall_clearance(20.0, 45.0, 10.0, 7.0)
    @test isapprox(h, 7.5475; atol=1e-12); @test state == "rising";
   @test clears
    h, state, clears = wall_clearance(20.0, 45.0, 30.0, 8.0)
    @test state == "falling"; @test !clears
    @test wall_clearance(10.0, 0.0, 0.0, 0.0) == (0.0, "falling", true)
    @test_throws ArgumentError wall_clearance(-1.0, 30.0, 2.0, 1.0)
    @test_throws ArgumentError wall_clearance(10.0, 30.0, 2.0, -1.0)
    @test_throws ArgumentError wall_clearance(10.0, 90.0, 2.0, 1.0)
    @test_throws ArgumentError wall_clearance(true, 30.0, 2.0, 1.0)
end
```